

Python for Data Science

Buổi 4: Data Loading, Storage, and File Formats

PGS-TS Nguyễn Mạnh Hùng

August 18, 2026

Cấu trúc Buổi 4

- 1 Ôn tập Buổi 3 và đặt vấn đề
- 2 Module 13: Đọc dữ liệu văn bản nâng cao (CSV/TXT)
- 3 Module 14: Ghi dữ liệu CSV & Xử lý thủ công
- 4 Module 15: JSON, HTML và XML
- 5 Module 16: Định dạng nhị phân & Nguồn dữ liệu bên ngoài

Lưu ý phạm vi

Buổi học tập trung kỹ năng **nạp và lưu** dữ liệu từ nhiều nguồn/định dạng khác nhau. Các phương pháp **xử lý** dữ liệu (missing data, merge, concat, groupby...) sẽ học từ Buổi 5.

Nhắc lại Buổi 3

- Series/DataFrame: ndarray gắn nhãn (index, columns).
- `pd.read_csv()` cơ bản: đọc file CSV chuẩn, có header, phân cách dấu phẩy.
- Bài tập với *US Baby Names*, *US President Heights* – đều là CSV "sạch", đúng chuẩn.

Câu hỏi mở đầu

Dữ liệu ngoài đời thực hiếm khi "sạch" như vậy: file không có header, dấu phân cách là khoảng trắng, giá trị thiếu được ghi bằng "NA", "NULL" hay để trống, file nặng vài GB, hoặc dữ liệu không phải CSV mà là JSON, HTML, Excel, hay nằm trong một cơ sở dữ liệu. Pandas xử lý những trường hợp này như thế nào?

Bản đồ định dạng dữ liệu

Văn bản
CSV, TXT

Bán cấu trúc
JSON, HTML, XML

Nhị phân
Pickle, Parquet, Excel, HDF5

Bên ngoài
Web API, Database

Bốn nhóm định dạng này bao phủ gần như mọi nguồn dữ liệu một Data Scientist gặp trong thực tế (McKinney, 2022, Ch.6) – lần lượt tương ứng với Module 13–16 của buổi học.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Đọc file không có header, hoặc có phân cách không chuẩn.
- 2 Dùng `index_col` để tạo index (kể cả MultiIndex) ngay khi đọc dữ liệu.
- 3 Xử lý dữ liệu thiếu được ghi theo nhiều quy ước khác nhau.
- 4 Đọc file lớn theo từng phần bằng `chunksize`.

File không có header

```
# ex2.csv (không có dòng header)
# 1,2,3,4,hello
# 5,6,7,8,world
# 9,10,11,12,foo
```

```
pd.read_csv("ex2.csv", header=None)
#    0    1    2    3    4
# 0  1    2    3    4  hello
# ...
```

```
names = ["a", "b", "c", "d", "message"]
pd.read_csv("ex2.csv", names=names)          # tu dat ten cot
pd.read_csv("ex2.csv", names=names, index_col="message") # dung 1 cot lam index
```

Ví dụ dùng đúng file ex2.csv từ notebook Chương 6 – *Data Loading, Storage, and File Formats* (McKinney, 2022).

index_col với nhiều cột: MultiIndex ngay khi đọc

```
# csv_mindex.csv
# key1,key2,value1,value2
# one,a,1,2
# one,b,3,4
# two,a,9,10 ...

parsed = pd.read_csv("csv_mindex.csv", index_col=["key1", "key2"])
print(parsed)
```

		value1	value2
key1	key2		
one	a	1	2
	b	3	4
two	a	9	10

Truyền một *danh sách* cột cho index_col tạo ngay MultiIndex – kỹ thuật này sẽ dùng lại nhiều ở Buổi 5 (Hierarchical Indexing).

Phân cách không cố định: sep với regex

```
# ex3.txt (phan cach bang so luong khoang trang KHONG co dinh)
#           A           B           C
# aaa -0.264438 -1.026059 -0.619500
# bbb  0.927272  0.302904 -0.032399
```

```
result = pd.read_csv("ex3.txt", sep=r"\s+")
print(result)
#           A           B           C
# aaa -0.264438 -1.026059 -0.619500
# bbb  0.927272  0.302904 -0.032399
```

Tham số sep nhận *biểu thức chính quy* (regex), không chỉ một ký tự cố định – `r"\s+"` khớp với một hoặc nhiều khoảng trắng liên tiếp.

Bỏ qua dòng không cần thiết: skiprows

```
# ex4.csv
# # hey!
# a,b,c,d,message
# # just wanted to make things more difficult for you
# # who reads CSV files with computers, anyway?
# 1,2,3,4,hello
# 5,6,7,8,world

pd.read_csv("ex4.csv", skiprows=[0, 2, 3])
#      a      b      c      d message
# 0  1      2      3      4  hello
# 1  5      6      7      8  world
```

Dữ liệu xuất ra từ các hệ thống thực tế (cảm biến, thiết bị đo, phần mềm export) thường có dòng chú thích/metadata xen giữa – skiprows loại bỏ chính xác các dòng đó theo chỉ số.

Dữ liệu thiếu được ghi theo nhiều quy ước

```
# ex5.csv
# something,a,b,c,d,message
# one,1,2,3,4,NA
# two,5,6,,8,world
# three,9,10,11,12,foo

result = pd.read_csv("ex5.csv")
print(result.isna())          # Pandas tu nhan dien "NA" va o trong la thieu du lieu

pd.read_csv("ex5.csv", na_values=["NULL"])          # them quy uoc rieng
sentinels = {"message": ["foo", "NA"], "something": ["two"]}
pd.read_csv("ex5.csv", na_values=sentinels, keep_default_na=False) # theo tung cot
```

Mỗi hệ thống/quốc gia có thể dùng ký hiệu riêng cho dữ liệu thiếu ("NA", "NULL", "?", -999...) – `na_values` cho phép khai báo tường minh, kể cả theo từng cột.

Đọc file lớn theo từng phần

```
pd.read_csv("ex6.csv", nrows=5)                # chỉ đọc 5 dòng đầu (xem trước)

chunker = pd.read_csv("ex6.csv", chunksize=1000) # đọc theo lô 1000 dòng
tot = pd.Series([], dtype="float64")
for piece in chunker:
    tot = tot.add(piece["key"].value_counts(), fill_value=0)
tot = tot.sort_values(ascending=False)
```

Vì sao cần chunksize?

Với file vài GB không vừa bộ nhớ RAM, chunksize cho phép xử lý *từng phần* rồi tổng hợp kết quả – nguyên lý nền tảng của xử lý dữ liệu lớn ngoài phạm vi RAM.

Tổng kết Module 13

- `header=None/names=`: đọc file không có dòng tiêu đề.
- `index_col`: chỉ định cột làm index, truyền list để tạo MultiIndex.
- `sep=`: nhận regex, không chỉ ký tự cố định.
- `skiprows`: bỏ qua các dòng chú thích/metadata.
- `na_values, keep_default_na`: khai báo quy ước dữ liệu thiếu.
- `nrows, chunksize`: đọc file lớn theo từng phần.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- 1 Ghi DataFrame ra CSV với các tùy chọn định dạng khác nhau.
- 2 Xử lý file CSV "khó" bằng module csv chuẩn của Python khi `read_csv` không đủ linh hoạt.

to_csv(): ghi dữ liệu với nhiều tùy chọn

```
data = pd.read_csv("ex5.csv")

data.to_csv("out.csv")                                # mac dinh: giu index

import sys
data.to_csv(sys.stdout, sep="|")                       # doi ky tu phan cach
data.to_csv(sys.stdout, na_rep="NULL")                 # doi cach ghi gia tri thieu
data.to_csv(sys.stdout, index=False, header=False)    # bo index va header
data.to_csv(sys.stdout, index=False, columns=["a", "b", "c"]) # chi ghi 1 so cot
```

to_csv() đối xứng với read_csv(): hầu hết tham số đọc đều có tham số ghi tương ứng (sep, na_rep, index, header, columns).

Khi read_csv() không đủ: module csv

```
# ex7.csv có dấu ngoặc kép quanh mọi trường
# "a","b","c"
# "1","2","3"
# "1","2","3"

import csv

with open("ex7.csv") as f:
    lines = list(csv.reader(f))

header, values = lines[0], lines[1:]
data_dict = {h: v for h, v in zip(header, zip(*values))}
print(data_dict)
# {'a': ('1', '1'), 'b': ('2', '2'), 'c': ('3', '3')}
```

Module csv chuẩn của Python (không thuộc Pandas) cho quyền kiểm soát chi tiết hơn – hữu ích với các file có định dạng "dị" (dấu quote lồng nhau, dấu phân cách đặc biệt) mà read_csv khó xử lý tự động.

Tổng kết Module 14

- `to_csv()`: đối xứng với `read_csv()` – `sep`, `na_rep`, `index`, `header`, `columns`.
- Module `csv` chuẩn: phương án dự phòng khi `read_csv()` không đủ linh hoạt.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- ➊ Chuyển đổi qua lại giữa JSON và DataFrame.
- ➋ Trích xuất bảng dữ liệu từ trang HTML bằng `read_html()`.
- ➌ Đọc dữ liệu XML bằng `read_xml()`.

JSON: đọc thủ công với json.loads

```
obj = """
{"name": "Wes",
 "cities_lived": ["Akron", "Nashville", "New York", "San Francisco"],
 "siblings": [{ "name": "Scott", "age": 34},
               { "name": "Katie", "age": 42}]}
"""

import json
result = json.loads(obj)          # JSON string -> dict/list long nhau

siblings = pd.DataFrame(result["siblings"], columns=["name", "age"])
print(siblings)
#    name  age
# 0  Scott   34
# 1  Katie   42
```

Dữ liệu JSON thường *lồng nhau* (nested) – cần chọn đúng phần tử (như `result["siblings"]`) trước khi đưa vào `pd.DataFrame()`, giống kỹ thuật "list of dict" đã học ở Buổi 3.

pd.read_json() / to_json()

```
# example.json
# [{"a": 1, "b": 2, "c": 3},
#  {"a": 4, "b": 5, "c": 6},
#  {"a": 7, "b": 8, "c": 9}]

data = pd.read_json("example.json")
print(data)
#    a  b  c
# 0  1  2  3
# 1  4  5  6
# 2  7  8  9

data.to_json()                # DataFrame -> JSON string (orient mặc định)
data.to_json(orient="records") # dạng list of dict - dễ đọc hơn
```

Khi JSON có cấu trúc "phẳng" (mảng các object đồng nhất), `pd.read_json()` tiện hơn nhiều so với `json.loads` thủ công.

HTML: trích xuất bảng bằng read_html()

```
tables = pd.read_html("fdic_failed_bank_list.html")
print(len(tables))          # 1 - trang chỉ có 1 bảng <table>

failures = tables[0]
print(failures.columns.tolist())
# ['Bank Name', 'City', 'ST', 'CERT',
#  'Acquiring Institution', 'Closing Date', 'Updated Date']

close_timestamps = pd.to_datetime(failures["Closing Date"])
close_timestamps.dt.year.value_counts()    # số ngân hàng phá sản theo năm
```

Ví dụ thật từ notebook Chương 6: danh sách các ngân hàng Mỹ phá sản (FDIC). `read_html()` tự động tìm và phân tích mọi thẻ `<table>` trong trang, trả về danh sách DataFrame.

XML: đọc dữ liệu bằng read_xml()

```
# datasets/mta_perf/Performance_MNR.xml (du lieu hieu suat tau MTA New York)
# <PERFORMANCE><INDICATOR>
#   <INDICATOR_NAME>On-Time Performance (West of Hudson)</INDICATOR_NAME>
#   <YTD_TARGET>95.00</YTD_TARGET>
#   <YTD_ACTUAL>96.90</YTD_ACTUAL>
# ...
```

```
perf = pd.read_xml("datasets/mta_perf/Performance_MNR.xml")
print(perf.shape)          # (648, 16)
print(perf[["INDICATOR_NAME", "YTD_TARGET", "YTD_ACTUAL"]].head())
```

Trước khi có `pd.read_xml()` (thêm từ Pandas 1.3), việc đọc XML cần dùng thư viện `lxml.objectify` và viết vòng lặp trích xuất từng thẻ thủ công – `read_xml()` rút gọn toàn bộ thành một lệnh.

Tổng kết Module 15

- JSON lồng nhau: `json.loads()` + chọn đúng phần tử trước khi tạo DataFrame.
- JSON "phẳng": `pd.read_json()/to_json()` tiện lợi hơn.
- `pd.read_html()`: trích xuất mọi bảng `<table>` từ HTML thành list DataFrame.
- `pd.read_xml()`: đọc trực tiếp file XML, không cần parse thủ công.

Chuẩn đầu ra (Learning Outcomes)

Sau module này, học viên có khả năng:

- ➊ Lưu/đọc DataFrame ở định dạng nhị phân (Pickle, Parquet, Excel).
- ➋ Giải thích khi nào nên dùng định dạng nhị phân thay vì CSV.
- ➌ Lấy dữ liệu từ Web API và truy vấn dữ liệu từ cơ sở dữ liệu SQL vào DataFrame.

Pickle: lưu nhanh, không portable

```
frame = pd.read_csv("ex1.csv")
frame.to_pickle("frame_pickle")

pd.read_pickle("frame_pickle")      # doc lai nhanh hon nhieu so voi CSV
```

Cảnh báo

Pickle chỉ đảm bảo đọc lại được trong **thời gian ngắn** và với **cùng phiên bản Pandas/Python** – không phù hợp để lưu trữ lâu dài hoặc chia sẻ giữa các máy khác nhau (khuyến cáo chính thức từ Pandas documentation).

Parquet: định dạng cột hiệu quả

```
# can cài đặt: pip install pyarrow
```

```
frame.to_parquet("frame.parquet")  
pd.read_parquet("frame.parquet")
```

Vì sao Parquet phổ biến trong công nghiệp?

Parquet lưu dữ liệu theo **cột** (columnar), nén tốt, đọc nhanh hơn CSV nhiều lần với dữ liệu lớn, giữ nguyên kiểu dữ liệu (không như CSV – luôn là văn bản thuần túy) – định dạng chuẩn trong các hệ thống Big Data (Spark, Data Lake).

Excel: ExcelFile, read_excel, to_excel

```
xlsx = pd.ExcelFile("ex1.xlsx")  
print(xlsx.sheet_names)                # liệt kê các sheet  
  
frame = xlsx.parse(sheet_name="Sheet1")    # đọc 1 sheet cụ thể  
frame = pd.read_excel("ex1.xlsx", sheet_name="Sheet1")  # cách đọc trực tiếp  
  
with pd.ExcelWriter("output.xlsx") as writer:    # ghi ra file Excel  
    frame.to_excel(writer, sheet_name="Sheet1")
```

ExcelFile hữu ích khi cần đọc *nhiều sheet* từ cùng một file mà không phải mở lại file nhiều lần.

HDF5: lưu trữ dữ liệu lớn ngoài RAM

Khái niệm

HDF5 (Hierarchical Data Format) là định dạng nhị phân cho phép lưu **nhiều DataFrame** trong một file, hỗ trợ truy vấn một phần dữ liệu mà không cần nạp toàn bộ vào RAM.

```
store = pd.HDFStore("mydata.h5")
store["obj1"] = frame                                # lưu 1 DataFrame với 1 "key"
store.put("obj2", frame, format="table")             # định dạng hỗ trợ truy vấn
store.select("obj2", where=["index >= 10"])          # chỉ đọc 1 phần dữ liệu
```

Ít dùng hơn Parquet trong các dự án mới, nhưng vẫn phổ biến trong khoa học dữ liệu/khoa học tự nhiên (dữ liệu cảm biến, mô phỏng).

Web API: lấy dữ liệu bằng requests

```
import requests

url = "https://api.github.com/repos/pandas-dev/pandas/issues"
resp = requests.get(url)
data = resp.json()          # JSON response -> list of dict

issues = pd.DataFrame(
    data, columns=["number", "title", "labels", "state"]
)
print(issues.head())
```

Quy trình chuẩn: gọi API bằng requests → .json() để lấy dữ liệu dạng Python (list/dict)
→ đưa vào pd.DataFrame() – đúng kỹ thuật "list of dict" đã học từ Buổi 3.

Database: SQL với sqlite3 và pandas

```
import sqlite3

con = sqlite3.connect("mydata.sqlite")
con.execute("""CREATE TABLE test
    (a VARCHAR(20), b VARCHAR(20), c REAL, d INTEGER);""")
con.executemany("INSERT INTO test VALUES(?, ?, ?, ?)", data)
con.commit()

cursor = con.execute("SELECT * FROM test")
rows = cursor.fetchall()
pd.DataFrame(rows, columns=[x[0] for x in cursor.description])

# Cach gon hon, dung sqlalchemy:
import sqlalchemy as sqla
db = sqla.create_engine("sqlite:///mydata.sqlite")
pd.read_sql("SELECT * FROM test", db)          # truc tiep ra DataFrame
```

Khi nào dùng định dạng nào?

Định dạng	Khi nào nên dùng	Tốc độ/Kích thước
CSV/TXT	Chia sẻ, đọc bằng Excel/mắt người, dữ liệu nhỏ-vừa	Chậm, nặng
JSON	Dữ liệu lồng nhau, Web API	Trung bình
Excel	Người dùng nghiệp vụ, báo cáo	Chậm
Pickle	Cache tạm thời, cùng máy/phiên bản	Rất nhanh
Parquet	Dữ liệu lớn, pipeline production	Nhanh, nhẹ
SQL Database	Dữ liệu quan hệ, truy vấn linh hoạt	Tùy hệ quản trị

Tổng kết Module 16

- Pickle: nhanh nhưng không portable – chỉ dùng tạm thời.
- Parquet: định dạng cột, hiệu quả cho dữ liệu lớn (cần pyarrow).
- Excel: `ExcelFile/read_excel/to_excel` – cho người dùng nghiệp vụ.
- HDF5: lưu trữ nhiều DataFrame, truy vấn một phần dữ liệu ngoài RAM.
- `requests + .json()`: lấy dữ liệu từ Web API.
- `sqlite3/sqlalchemy + pd.read_sql()`: truy vấn cơ sở dữ liệu SQL.

Bảng tổng hợp hàm I/O trong Buổi 4

Định dạng	Đọc	Ghi
CSV/TXT	<code>read_csv</code>	<code>to_csv</code>
JSON	<code>read_json</code> , <code>json.loads</code>	<code>to_json</code> , <code>json.dumps</code>
HTML	<code>read_html</code>	<code>to_html</code>
XML	<code>read_xml</code>	<code>to_xml</code>
Excel	<code>read_excel</code> , <code>ExcelFile</code>	<code>to_excel</code> , <code>ExcelWriter</code>
Pickle	<code>read_pickle</code>	<code>to_pickle</code>
Parquet	<code>read_parquet</code>	<code>to_parquet</code>
HDF5	<code>read_hdf</code> , <code>HDFStore</code>	<code>to_hdf</code>
SQL	<code>read_sql</code>	<code>to_sql</code>

Bài tập 1: Dữ liệu XML thật – MTA Performance

Dữ liệu: `python-for-data-analysis/datasets/mta_perf/Performance_MNR.xml` (648 bản ghi hiệu suất tàu Metro-North Railroad, New York).

- 1 Đọc file bằng `pd.read_xml()`; kiểm tra `shape`, `columns`, `dtypes`.
- 2 Dùng boolean indexing (đã học Buổi 3) lọc các chỉ số có `YTD_ACTUAL >= YTD_TARGET` (đạt chỉ tiêu).
- 3 Với cột `INDICATOR_NAME`, tìm chỉ số nào xuất hiện nhiều lần nhất trong dữ liệu (gợi ý: `value_counts()`).
- 4 So sánh thời gian đọc bằng `pd.read_xml()` so với việc parse thủ công bằng `lxml.objectify` (tham khảo notebook Chương 6).

Bài tập 2: Kết hợp nhiều định dạng trong một pipeline

- 1 Đọc notebooks/data/president_heights.csv (Handbook, Buổi 3) bằng `read_csv()`.
- 2 Ghi lại thành 3 định dạng khác nhau: `to_json(orient="records")`, `to_excel()`, `to_pickle()`.
- 3 Đọc lại cả 3 file vừa tạo, kiểm tra dtypes có thay đổi so với bản CSV gốc không (gợi ý: chú ý cột `height(cm)` sau khi qua JSON).
- 4 Tạo một SQLite database nhỏ, nạp dữ liệu tổng thống vào một bảng, rồi dùng `pd.read_sql()` truy vấn "các tổng thống cao hơn 185cm".

Câu hỏi thảo luận

Vì sao dtype của một cột số đôi khi bị thay đổi sau khi ghi ra JSON rồi đọc lại? Định dạng nào trong bài tập giữ nguyên dtype chính xác nhất?

Tài liệu tham khảo I

- McKinney, W. (2022). *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter* (3rd ed., Ch.6 – Data Loading, Storage, and File Formats). O'Reilly Media.
- McKinney, W. (2010). Data structures for statistical computing in Python. *Proceedings of the 9th Python in Science Conference*, 51–56.
- Pugh, D. R. (n.d.). *python-for-data-analysis* [Kho mã nguồn – notebook module-1/ch06.ipynb, dữ liệu examples/, datasets/mta_perf/]. GitHub.
<https://github.com/davidrpugh/python-for-data-analysis>
- VanderPlas, J. (2016). *Python Data Science Handbook: Essential Tools for Working with Data*. O'Reilly Media.
- The Apache Software Foundation. (2013). *Apache Parquet* [Software]. <https://parquet.apache.org>
- The HDF Group. (1997–2024). *Hierarchical Data Format, version 5*.
<https://www.hdfgroup.org/solutions/hdf5>
- The pandas development team. (2020). *pandas-dev/pandas: Pandas (Version 1.0)* [Software]. Zenodo.
<https://doi.org/10.5281/zenodo.3509134>

Chuẩn bị cho buổi tiếp theo: Pandas Nâng Cao I – Missing Data & Kết Hợp Dữ Liệu

Thông điệp

Biết nạp dữ liệu từ mọi định dạng chỉ là bước khởi đầu. Buổi 5 sẽ xử lý những gì xảy ra *sau khi* dữ liệu đã nằm trong DataFrame: giá trị thiếu, chỉ mục nhiều cấp, và kết hợp nhiều nguồn dữ liệu lại với nhau.